Spis treści

Wstęp **4**
1. Zarys projektu **5**
1.1. Motywacje utworzenia projektu 5
1.2. Cele i założenia projektu 6
1.3. Wykorzystane technologie i architektura systemu 7
1.4. System kontroli wersji i organizacja pracy 8
2. Analiza rynku **9**
2.1. System Mol Net+ 10
2.2. System Librus eBiblioteka (e-Biblio) 12
2.3. System Prolib 14
2.4. Podsumowanie porównania 16
3. Wymagania funkcjonalne i niefunkcjonalne **17**
3.1 Wymagania funkcjonalne 17
3.2. Wymagania niefunkcjonalne 19
4. Projektowanie systemu **20**
4.1 Diagramy przypadków użycia 20
4.2 Prototyp systemu - katalog biblioteki 22
5. Struktura bazy danych **25**
5.1 Zarządzanie zasobami 25
5.2 Zarządzanie użytkownikami 26
6. Implementacja projektu **28**
6.1 Implementacja API 28
6.1.1. Architektura i podział na warstwy logiczne 28
6.1.2. Bezpieczeństwo i autoryzacja 30
6.1.3. Architektura punktów końcowych (Endpoints) 31
6.1.4. Przykład implementacji dla dodawania książki 32
6.2 Implementacja warstwy frontend 35
6.2.1. Architektura Blazor 35
6.2.2. Przykład implementacji strony katalogu / wyników wyszukiwania 36
6.3 Implementacja czatbota AI 40
6.3.1. Konfiguracja środowiska chmurowego 40
6.3.2. Logika biznesowa i integracja 42
6.3.3. Interfejs czatu 45
6.3.4. Przykład konwersacji z agentem AI 48
6.4 Opcje dostępności i inkluzywność systemu 49
6.4.1. Personalizacja i mechanizm trwałego zapisu ustawień 49
6.4.2. Semantyka i technologie asystujące 50
6.4.3. Weryfikacja i audyt dostępności z użyciem Accessibility Insights 50
7. Testy użyteczności systemu **51**
7.1 Scenariusze testowe użyteczności systemu 51
7.2 Wyniki testów użyteczności 52
7.3 Wykryte błędy i poprawki 53
7.4 Ocena System Usability Scale (SUS) 53
8. Podsumowanie projektu i wnioski końcowe **54**
8.1 Weryfikacja realizacji celu pracy i założeń projektowych 54
8.2 Ocena przyjętych rozwiązań technologicznych 54
8.3 Podsumowanie i kierunki dalszego rozwoju 54
Spis ilustracji, fragmentów kodu, tabel i diagramów **57**
Streszczenie pracy dyplomowej **58**
Diploma Thesis Summary **58**

1.2. Cele i założenia projektu

Celem projektu było utworzenie systemu webowego do obsługi szkolnej biblioteki i możliwości przeglądania zasobów online, umożliwiającego bezproblemowe rozbudowanie go w przyszłości. System składa się z dwóch podsystemów - katalogu biblioteki oraz panelu zarządzania, każdy z dedykowanym interfejsem użytkownika. Dostęp do danego systemu i jego funkcjonalności zależy od roli użytkownika, sam katalog jest dostępny bez logowania, ale ograniczając pewne funkcjonalności (np. rezerwacja książki).

Aplikacja jest dostosowana pod czytniki ekranowe oraz oferuje opcje dostępności zgodne z WCAG 2.1[9] i opcje ułatwiające poruszanie się po stronie, takie jak:

1) zmiana wielkości czcionki: mała (12px, średnia (16px; domyślnie), duża (20px);
2) zmiana motywu strony: zielony (domyślnie), niebieski, brązowy, ciemnoszary;
3) wybór trybu wysokiego kontrastu strony;
4) mapa strony;
5) instrukcja użytkowania katalogu;

Czytelnik może przeglądać i wyszukiwać książki, dodawać je do ulubionych, dodawać je do koszyka oraz je wypożyczać (jeśli są dostępne). Katalog oferuje także przegląd polecanych przez bibliotekę pozycji oraz osobne podstrony dla listy lektur i podręczników. Czytelnik może także uzyskać pomoc od czatbota, korzystającego ze wsparcia AI.

Panel zarządzania dla bibliotekarza umożliwia przegląd, dodawanie, modyfikowanie i usuwanie zasobów (książek, podręczników, lektur, egzemplarzy) oraz kont czytelników. Dostęp do panelu jest przyznawany użytkownikom z rolą bibliotekarza lub administratora, a przełączanie między panelem i katalogiem odbywa się przy pomocy specjalnego przycisku w pasku nawigacji.

Zakresem projektu jest:

a) Utworzenie bazy danych i połączenie jej z aplikacją za pomocą API
b) Utworzenie osobnej warstwy frontend i backend oraz scalenie ich w jedną aplikację
c) Utworzenie systemu autoryzacji i logowania użytkowników
d) Utworzenie strony katalogu, na której użytkownik może przeglądać i rezerwować książki oraz zapisywać je do ulubionych, a także przeglądać historię swoich wypożyczeń
e) Utworzenie strony panelu zarządzania, na której autoryzowany użytkownik może zarządzać zasobami oraz użytkownikami (przegląd, dodawanie, modyfikacja, usuwanie)
f) Implementacja opcji dostępności zgodnych z wytycznymi WCAG 2.1
g) Implementacja asystenta czytelnika w postaci czatu AI
h) Implementacja responsywnego, przyjaznego i czytelnego interfejsu użytkownika
i) Zastosowanie nowoczesnych wzorców projektowych

[9]https://www.gov.pl/web/dostepnosc-cyfrowa/wcag-21-w-skrocie

1.3. Wykorzystane technologie i architektura systemu

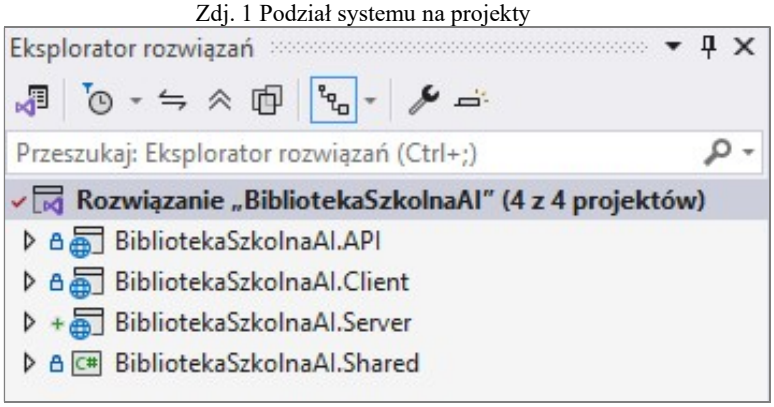
Fundamentem projektu jest platforma .NET 9. Architektura systemu została zaprojektowana jako rozwiązanie wielowarstwowe i rozproszone, co zapewnia skalowalność oraz łatwość utrzymania. Struktura rozwiązania opiera się na ścisłym podziale na niezależne warstwy logiczne:

1) **Warstwa Prezentacji (Frontend):** Aplikacja kliencka zrealizowana w technologii *Blazor Web App*.
2) **Warstwa Interfejsu (API)**: RESTful API odpowiedzialne za obsługę żądań HTTP.
3) **Warstwa Logiki Biznesowej (Services)**: Moduł przetwarzający reguły biznesowe i walidację.
4) **Warstwa Dostępu do Danych (Repositories)**: Bezpośrednie operacje na bazie danych.

Fizyczna struktura solucji została podzielona na 4 projekty (**Zdj. 1**):

1) **.API**: Główny projekt backendowy, implementujący warstwę logiki biznesowej, dostęp do danych oraz wystawiający punkty końcowe REST API.
2) **.Client**: Projekt warstwy prezentacji, zawierający komponenty interfejsu użytkownika uruchamiane w przeglądarce.
3) **.Server**: Projekt pełniący rolę hosta aplikacji, odpowiedzialny za konfigurację żądań, rejestrację serwisów *Dependency Injection*[10] (DI) oraz serwowanie aplikacji klienckiej.
4) **.Shared**: Biblioteka współdzielona, zawierająca definicje obiektów transferu danych (DTO[11]), typy wyliczeniowe (enum) oraz atrybuty walidacji.

Zdj. 1 Podział systemu na projekty



Źródło 2 Opracowanie własne w Visual Studio 2022

Rdzeń systemu stanowi asynchroniczne API oparte na frameworku ASP.NET Core 9. Wewnątrz projektu .API warstwa serwisów centralizuje logikę biznesową, a repozytoria izolują operacje na bazie danych.

Fundamentem składowania danych jest silnik *Microsoft SQL Server*. Schemat bazy wymodelowano w narzędziu *Redgate Data Modeler*, co zapewniło poprawność relacji na etapie planowania. Dostęp do danych realizuje *Entity Framework Core 9*. Zaimplementowano również *Global Query Filters* do zarządzania dostępnością rekordów oraz *Split Queries* w celu optymalizacji wydajności zapytań pobierających powiązane dane.

Warstwa prezentacji została zrealizowana w modelu Blazor Web App (.NET 9), wykorzystującym nowoczesny model renderowania hybrydowego Server-Side Rendering (SSR)

[10] Wstrzykiwanie zależności - polega na dostarczaniu klasie niezbędnych do działania obiektów z zewnątrz, zamiast tworzenia ich bezpośrednio w jej kodzie.
[11] DTO - Data Transfer Object, obiekt transferu danych

i Client-Side (WebAssembly). Aplikacja Blazor Server pełni rolę pośrednika (Proxy), przekazując żądania z przeglądarki do właściwego API Backendowego, co zwiększa bezpieczeństwo architektury.

Interfejs użytkownika (UI) zaprojektowano z wykorzystaniem makiet w narzędziu *Figma*. Do implementacji interfejsu użyto biblioteki *Bootstrap* dla stworzenia responsywnego układu (RWD), oraz pakietu *Syncfusion Blazor Components*. Pakiet *Syncfusion* posłużył do budowy zaawansowanych tabel z obsługą sortowania, filtrowania i paginacji po stronie serwera, a także do obsługi formularzy, wykresów oraz okien modalnych.

Kluczowym komponentem projektu jest moduł inteligentnego asystenta, zbudowany w oparciu o platformę *Azure AI Foundry*. Rozwiązanie to implementuje zaawansowaną architekturę RAG (Retrieval-Augmented Generation[12]) w modelu hybrydowym, integrując dwa zróżnicowane źródła danych w celu dostarczenia precyzyjnych odpowiedzi:

a) **Dane dynamiczne**: Asystent komunikuje się z API systemu w celu pobierania bieżących informacji o stanie katalogu (np. dostępność egzemplarzy).

b) **Dane statyczne**: Wykorzystano usługę *Azure AI Search* do indeksowania statycznej bazy wiedzy (plik DOCX), zawierającej informacje instytucjonalne, procedury i instrukcje.

Faza analityczna projektu obejmowała modelowanie systemu w narzędziu *Visual Paradigm*, w ramach którego opracowano diagramy przypadków użycia i diagramy sekwencji.

1.4. System kontroli wersji i organizacja pracy

Zarządzanie kodem źródłowym oraz historią zmian w projekcie realizowano przy użyciu rozproszonego systemu kontroli wersji Git. Jako platformę hostingową dla zdalnego repozytorium wybrano serwis GitHub, co umożliwiło bezpieczne przechowywanie kodu w chmurze oraz dostęp do narzędzi wspierających proces wytwórczy.

Organizacja pracy z kodem opierała się na strategii gałęzi. Nowe funkcjonalności oraz poprawki implementowano na wydzielonych gałęziach roboczych, a następnie - po weryfikacji poprawności działania - scalano je z główną gałęzią projektu. Podejście to pozwoliło na zachowanie stabilności wersji produkcyjnej aplikacji oraz umożliwiło łatwe śledzenie postępów prac (historia zmian).

Kluczowym elementem konfiguracji repozytorium jest plik .gitignore, dostosowany specyficznie do środowiska .NET. Został on skonfigurowany tak, aby wykluczyć z wersjonowania pliki binarne (katalogi bin i obj), pliki tymczasowe środowiska Visual Studio oraz lokalne pliki konfiguracyjne zawierające dane wrażliwe. Dzięki temu zachowano czystość repozytorium oraz bezpieczeństwo haseł i kluczy API.

Ostateczną wersję projektu opublikowano w osobnym repozytorium pod linkiem: https://github.com/iguanaiza/praca-inzynierska-biblioteka-ai

[12] https://learn.microsoft.com/en-us/azure/ai-foundry/concepts/retrieval-augmented-generation?view=foundry-classic

6. Implementacja projektu

6.1 Implementacja API

6.1.1. Architektura i podział na warstwy logiczne

W projektowaniu API zastosowano architekturę wielowarstwową z wyraźnym podziałem na warstwy logiczne. Dzięki wykorzystaniu wbudowanego *Dependency Injection*, poszczególne komponenty są luźno powiązane, co ułatwia testowanie i przyszły rozwój systemu. Struktura przetwarzania pojedynczego żądania HTTP przebiega według schematu:
Kontroler → Serwis → Repozytorium → Baza Danych.

1. **Kontrolery** pełnią rolę punktów wejścia (Entry Points) i nie zawierają logiki biznesowej. Ich zadaniem jest:
   a) Odebranie żądania http,
   b) Wstępna walidacja modelu,
   c) Przekazanie danych do odpowiedniego serwisu,
   d) Zwrócenie odpowiedniego kodu statusu HTTP (np. 200 OK, 400 Bad Request) wraz ze sformatowaną odpowiedzią (JSON).
2. **Serwisy** implementują całą logikę biznesową aplikacji. Serwisy odpowiadają za:
   a) Walidację reguł biznesowych,
   b) Przetwarzanie danych,
   c) Mapowanie obiektów domenowych na DTO,
   d) Komunikację z repozytoriami.
3. **Repozytoria** odpowiadają za abstrakcję dostępu do danych, tj. ukrywają szczegóły implementacyjne *EF Core*, dostarczając metody takie jak *GetById*, *Add* czy *Update*. Dodatkowo zastosowano optymalizacje takie jak *AsNoTracking()* dla operacji odczytu oraz *AsSplitQuery()* dla zapytań pobierających złożone kolekcje, co zapobiega problemowi generowania ogromnej liczby wierszy (tzw. "Eksplozja kartezjanizmu")[19].

Interfejs API został zaprojektowany zgodnie z zasadami REST. Wykorzystano standardowe metody HTTP do odzwierciedlenia intencji operacji:

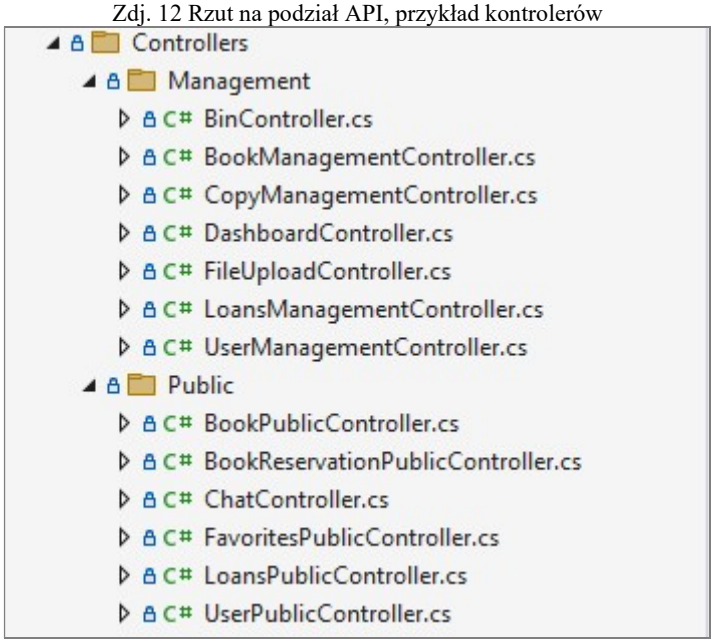
a) **GET**: Pobieranie danych,
b) **POST**: Tworzenie nowych zasobów,
c) **PUT**: Aktualizacja istniejących zasobów,
d) **DELETE**: Trwałe usuwanie zasobów.

Do testowania API wykorzystano bibliotekę *Swagger* (OpenAPI). Pozwala ona na interaktywne sprawdzanie endpointów, włącznie z obsługą autoryzacji JWT[20], bezpośrednio z poziomu przeglądarki.

API zostało podzielone na dwa segmenty: obsługę katalogu i obsługę systemu zarządzania (**Zdj. 12**). Katalog jest dostępny dla wszystkich użytkowników, w tym niezalogowanych. Niektóre funkcjonalności (np. wypożyczanie książek) są dostępne tylko po zalogowaniu. System zarządzania dostępny jest jedynie po zalogowaniu dla użytkowników z rolą bibliotekarza lub administratora.

[19] https://learn.microsoft.com/pl-pl/ef/core/querying/single-split-queries
[20] JWT - JSON Web Token

Zdj. 12 Rzut na podział API, przykład kontrolerów



Źródło 23 Opracowanie własne

System nie eksponuje na zewnątrz encji bazodanowych, co jest kluczową praktyką bezpieczeństwa i separacji. Zamiast tego wykorzystano wzorzec Data Transfer Object. Dla każdego zasobu zdefiniowano osobne modele do odczytu (np. *BookGetDetailedDto*) oraz do zapisu (np. *BookCreateDto*). Do automatycznej transformacji (mapowania) obiektów między encją a DTO wykorzystano bibliotekę *AutoMapper*[21]. Pozwoliło to na redukcję kodu i centralizację logiki projekcji danych w pliku *MappingProfile.cs* (Kod **1**).

Kod 1 Fragment implementacji AutoMapper

```
public class MappingProfile : Profile
{
    public MappingProfile()
    {
        CreateMap<Book, BookGetBaseDto>()
            .ForMember(d => d.BookAuthor, o => o.MapFrom(s =>
             FormatAuthor(s.BookAuthor)))
            .ForMember(d => d.BookPublisher, o => o.MapFrom(s =>
             s.BookPublisher.Name))
            .ForMember(d => d.AvailableCopyCount, o => o.MapFrom(s =>
             s.AvailableCopyCount));
        […]
        CreateMap<Book, BookGetForCatalogDto>()
            .IncludeBase<Book, BookGetForListDto>()
            .ForMember(d => d.BookCategory, o => o.MapFrom(s => s.BookCategory.Name))
            .ForMember(d => d.BookType, o => o.MapFrom(s => s.BookType.Title))
            .ForMember(d => d.BookSeries, o => o.MapFrom(s => s.BookSeries != null ?
             s.BookSeries.Title : null))
            .ForMember(d => d.BookGenres, o => o.MapFrom(s => s.BookBookGenres.Select
             (x => x.BookGenre.Title).ToList()))
            .ForMember(d => d.BookSpecialTags, o => o.MapFrom(s =>
             s.BookBookSpecialTags.Select(x => x.BookSpecialTag.Title).ToList()))
            .ForMember(d => d.FavoriteCount, o => o.MapFrom(s => s.FavoriteCount))
            .ForMember(d => d.IsFavorite, o => o.MapFrom(s => false));
        […]
```

Źródło 24 Opracowanie własne

[21] https://www.nuget.org/packages/automapper/

6.2 Implementacja warstwy frontend

6.2.1. Architektura Blazor

Warstwa interfejsu użytkownika została zrealizowana w technologii *Blazor*, co pozwoliło na wykorzystanie języka C# oraz składni Razor do budowy dynamicznych, interaktywnych widoków. Architektura frontendu opiera się na modularnych blokach zwanych komponentami. Komponent to samodzielna część interfejsu użytkownika (UI) z logiką przetwarzania umożliwiającą dynamiczne działanie[23].

Istotnym aspektem budowy komponentów jest również zarządzanie zasobami i pamięcią. Mimo, że środowisko .NET wykorzystuje automatyczny mechanizm *Garbage Collector* [24], w aplikacjach typu SPA kluczowe jest zapobieganie wyciekom pamięci. W tym celu, komponenty korzystające z długotrwałych subskrypcji zdarzeń lub zewnętrznych zasobów, implementują interfejs *IDisposable*[25](np. powiadomienia akcji, wyszukiwarka). Pozwala to na ręczne zwolnienie zasobów i wyrejestrowanie zdarzeń w momencie niszczenia komponentu, co zapewnia stabilność aplikacji.

Aby zapewnić spójność wizualną i nawigacyjną w obrębie całej aplikacji, zastosowano mechanizm szablonów (*Layouts*). System wykorzystuje kilka takich szablonów, z podziałem na katalog, system zarządzania oraz komponenty wspólne. *Szablon* to komponent Razor, który współdzieli znaczniki z komponentami, które się do niego odwołują. Szablony mogą korzystać z powiązań danych, wstrzykiwania zależności i innych funkcji komponentów[26].

Warstwa wizualna aplikacji została zaprojektowana w oparciu o responsywny system siatki frameworku Bootstrap 5, wzbogacony o niestandardowe definicje stylów. Zastosowano hybrydowe podejście do zarządzania arkuszami stylów:

a) **Style globalne (app.css)**: Główny arkusz stylów umieszczony w projekcie serwera. Definiuje on zmienne CSS dla palety kolorów (motyw biblioteki), typografię, oraz wygląd wspólnych elementów (np. przycisków, nagłówków kart).

b) **Izolacja stylów**: Dla komponentów wymagających specyficznego wyglądu (nie wpływającego na resztę aplikacji) wykorzystano mechanizm *Scoped CSS*. Pozwala to na uniknięcie konfliktów nazw klas i utrzymanie czystości kodu.

c) **Biblioteki UI**: Komponenty zewnętrzne, takie jak *Syncfusion Blazor*, ładują własne style bazowe, które są nadpisywane przez globalny *app.css* w celu dopasowania do spójnego motywu graficznego aplikacji.

W kolejnym podrozdziale opisano implementację warstwy frontend na przykładzie strony katalogu, będącej jednocześnie stroną z wynikami wyszukiwania.

---

[23] https://learn.microsoft.com/pl-pl/aspnet/core/blazor/compo-nents/?view=aspnetcore-9.0
[24] https://learn.microsoft.com/pl-pl/dotnet/standard/garbage-collection/fundamentals
[25] https://learn.microsoft.com/pl-pl/dotnet/api/system.idisposable?view=net-9.0
[26] https://learn.microsoft.com/pl-pl/aspnet/core/blazor/components/ layouts?view=aspnetcore-9.0

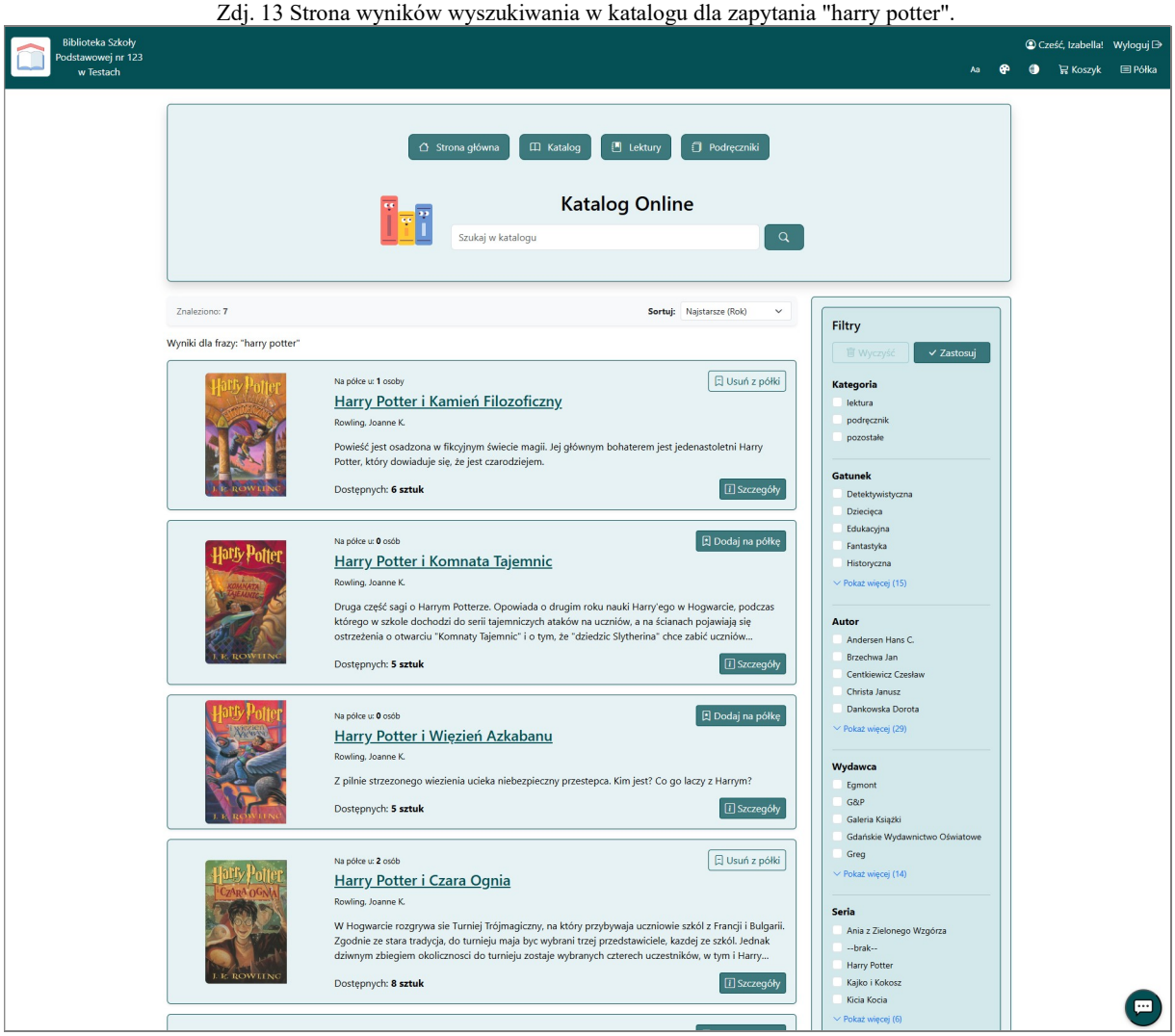
6.2.2. Przykład implementacji strony katalogu / wyników wyszukiwania

Komponent *Catalog.razor* stanowi centrum modułu bibliotecznego, obsługując dwie kluczowe ścieżki routingu: */katalog* (przeglądanie zasobów) oraz */szukaj* (prezentacja wyników).

Całość osadzono w układzie *CatalogLayout*, który zapewnia spójną strukturę interfejsu - od górnego paska nawigacji i zintegrowanej wyszukiwarki, po okno czatbota AI i stopkę (**Błąd! Nie można odnaleźć źródła odwołania.**). Architektura widoku realizuje wzorzec *Single Page Application* (SPA), dzięki czemu filtrowanie wyników odbywa się płynnie, poprzez asynchroniczne odświeżanie danych, bez przeładowywania całej strony.

Warstwa wizualna, oparta o system siatki Bootstrap, zapewnia pełną responsywność. Panel filtrów (*FiltersPanelComponent*) na komputerach jest stale widoczny w sekcji bocznej (*<aside>*), natomiast na urządzeniach mobilnych chowa się w wysuwanym menu. Rekordy prezentowane są jako pionowa lista kart, zawierająca okładkę, autora, fragment opisu książki oraz licznik dostępnych egzemplarzy Zdj. **13**, Kod **7**).

Zdj. 13 Strona wyników wyszukiwania w katalogu dla zapytania "harry potter".



Źródło 31 Opracowanie własne

6.3 Implementacja czatbota AI

Moduł asystenta został zaprojektowany jako rozwiązanie hybrydowe, integrujące logikę biznesową aplikacji Blazor Server z możliwościami generatywnej sztucznej inteligencji dostępnej w chmurze Microsoft Azure. Taka architektura pozwala na połączenie swobodnej konwersacji w języku naturalnym z dostępem do aktualnych danych bibliotecznych w czasie rzeczywistym.

Implementacja czatbota była procesem wieloetapowym. Prace rozpoczęto od konfiguracji modelu językowego i zdefiniowania jego zachowań w Azure AI Fondury. Następnie zbudowano warstwę komunikacyjną w języku C#, odpowiedzialną za bezpieczną wymianę danych między aplikacją a agentem AI. Na koniec utworzono interfejs użytkownika, który umożliwia wygodną obsługę czatu w przeglądarce.

Przyjęto model konwersacji sesyjnej - identyfikator wątku (*ThreadId*) jest przechowywany w pamięci aplikacji jedynie na czas trwania aktywnej sesji użytkownika. Zamknięcie przeglądarki lub odświeżenie strony skutkuje zakończeniem bieżącego wątku i inicjalizacją nowej, czystej konwersacji przy ponownym kontakcie.

6.3.1. Konfiguracja środowiska chmurowego

Proces rozpoczęto od konfiguracji usługi *Azure AI Foundry*. W ramach projektu utworzono agenta opartego na modelu *gpt-4o-mini*. Wybór podyktowany był nie tylko optymalnym stosunkiem kosztów do wydajności, ale także specyfiką zadań. Dodatkowym atutem jest niska latencja modelu, co gwarantuje płynność rozmowy.

Kluczowym etapem było „strojenie" modelu (*Prompt Engineering*). Zdefiniowano personę asystenta bibliotecznego, narzucając ton wypowiedzi dostosowany do wieku uczniów szkoły podstawowej. Równolegle skonfigurowano warstwę filtrów bezpieczeństwa, która automatycznie blokuje treści nieodpowiednie (np. mowę nienawiści, przemoc).

Zaimplementowano również logiczną warstwę autokorekty, instruując model, aby przed wywołaniem akcji normalizował zapytania użytkownika i poprawiał błędy ortograficzne (np. zamieniając „hary poter" na „Harry Potter"). Dzięki temu parametry przekazywane do bazy danych SQL są zawsze poprawne syntaktycznie, co zwiększa skuteczność wyszukiwania.

W warstwie instrukcji systemowych zaimplementowano również reguły sterujące logiką biznesową i behawioralną agenta. Podzielono je na dwa główne priorytety:

a) **Obsługa procedur bibliotecznych (RAG):** W przypadku pytań o sposób wypożyczania, narzucono ścisły schemat oparty na wgranej dokumentacji (*Retrieval-Augmented Generation*). Zastosowano tu technikę *Negative Constraints* (zakazów), zabraniając modelowi używania domyślnych sformułowań typu „kliknij Wypożycz", na rzecz specyficznego dla tego systemu polecenia „Dodaj do koszyka". Zapobiega to halucynacjom modelu, który bez tej instrukcji mógłby sugerować typowe, lecz nieistniejące w tej aplikacji funkcje.

b) **Weryfikacja dostępności (Tool Use):** Wprowadzono bezwarunkowy wymóg użycia narzędzia *search_books* przy zapytaniach o książki, z jednoczesnym zakazem „nadgorliwości". Agent został poinstruowany, aby przy prostym pytaniu o dostępność podawał jedynie stan magazynowy i link, nie zasypując użytkownika niechcianymi instrukcjami logowania, dopóki ten o nie wyraźnie nie zapyta.

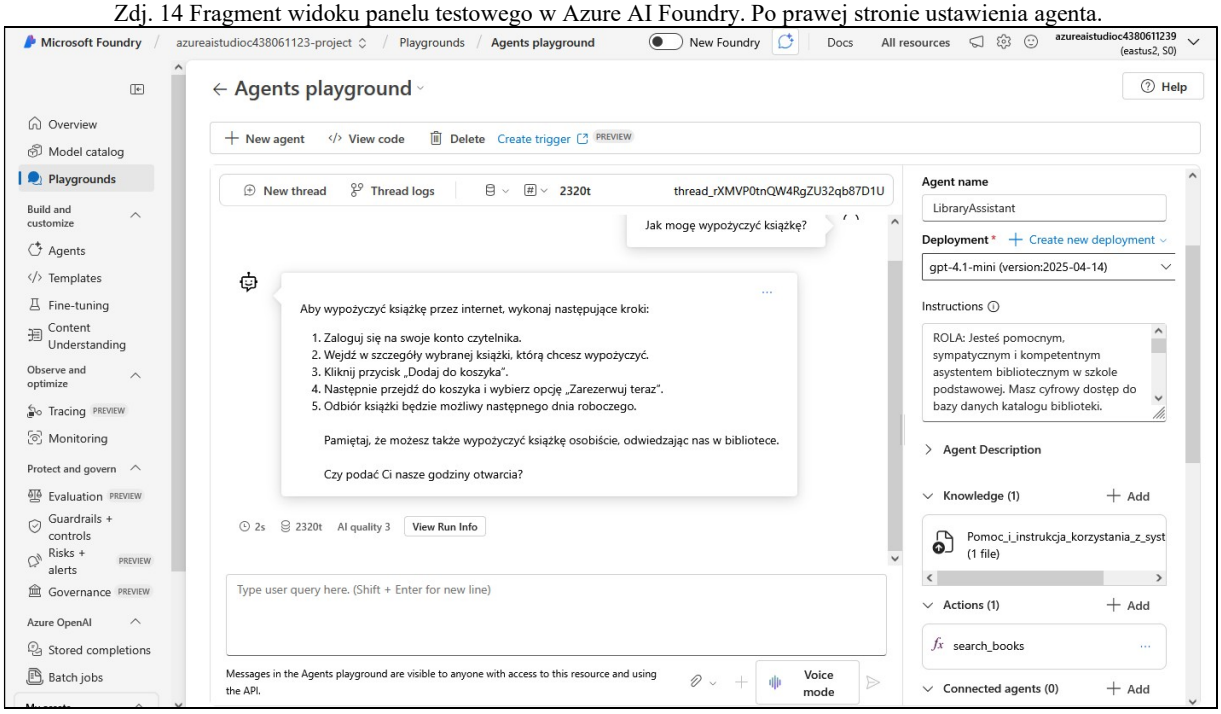
Dodatkowo zdefiniowano wytyczne dotyczące formatowania i stylu. Agent zobligowany jest do generowania interaktywnych łączy w formacie *Markdown*, integrując czat z routingiem aplikacji Blazor i umożliwiając płynną nawigację bez przeładowania strony. W warstwie językowej wdrożono zasadę neutralności płciowej (wymuszenie form bezosobowych, np. „Czy podać instrukcję?" zamiast „Czy chciałbyś…"), czyniąc asystenta inkluzywnym dla wszystkich uczniów. Całość dopełnia

mechanizm adaptacji stylu i języka wypowiedzi do rozmówcy (agent wykrywa i odpowiada w dowolnym języku np. polskim, angielskim, ukraińskim).

Integrację z backendem aplikacji zrealizowano w sposób programowy. Definicja funkcji *search_books* jest generowana bezpośrednio w kodzie C# i automatycznie synchronizowana z agentem w chmurze. Kod aplikacji dynamicznie buduje schemat JSON funkcji, precyzując nie tylko typy danych, ale również opisy semantyczne parametrów (np. informując model, że parametr *query* powinien zawierać znormalizowany tytuł książki). Podejście *Code-First* zapewnia spójność między logiką API a definicją narzędzi w Azure.

Weryfikacja zachowania agenta odbywała się w module *Playground*, co pozwoliło na przetestowanie scenariuszy decyzyjnych przed integracją z kodem (Zdj. **14**).

Zdj. 14 Fragment widoku panelu testowego w Azure AI Foundry. Po prawej stronie ustawienia agenta.



Źródło 35 Opracowanie własne w portalu Microsoft Foundry.

6.3.4. Przykład konwersacji z agentem AI

Przeprowadzony test potwierdził zdolność agenta do płynnego przełączania się między trzema niezależnymi źródłami informacji w ramach jednej sesji konwersacyjnej. Scenariusz wykazał poprawność działania następujących mechanizmów:

1) **Integracja z bazą danych i Autokorekta:** W pierwszej turze rozmowy system poprawnie zinterpretował zapytanie zawierające błędy językowe i literówki („hary poter"). Moduł normalizacji tekstu przekształcił frazę na poprawną formę, co umożliwiło skuteczne wywołanie funkcji *search_books* (SQL).

2) **Świadomość kontekstu i wiedza ogólna:** W odpowiedzi na pytanie „a o czym to jest", asystent wykorzystał mechanizm pamięci krótkotrwałej. Poprawnie powiązał zaimek wskazujący „to" z wcześniej wyszukaną książką. Odpowiedź została wygenerowana w oparciu o wbudowaną wiedzę ogólną modelu językowego, bez konieczności ponownego odpytywania bazy danych czy przeszukiwania plików.

3) **Wiedza proceduralna i RAG (Retrieval-Augmented Generation)**: W ostatniej fazie testu, na pytanie o sposób wypożyczania, system aktywował przeszukiwanie bazy statycznej (pliku .docx). Odpowiedź została skonstruowana ściśle według wgranej instrukcji biblioteki, używając terminologii tego konkretnego systemu bibliotecznego (nazwy przycisków). Potwierdzono przy tym działanie reguły neutralności płciowej - asystent konsekwentnie używał form bezosobowych, unikając narzucania rodzaju gramatycznego.

Zdj. 15 Testowa konwersacja z czatem AI na temat dostępności, tematyki i wypożyczenia książki.

Źródło 42 Opracowanie własne

6.4 Opcje dostępności i inkluzywność systemu

Projekt interfejsu aplikacji został opracowany w oparciu o projektowanie włączające (inkluzywne), mające na celu eliminację barier cyfrowych dla osób z niepełnosprawnościami wzrokowymi i poznawczymi. System spełnia kluczowe wytyczne standardu WCAG 2.1 na poziomie AA. Strategia wdrożenia dostępności objęła trzy filary: personalizację warstwy prezentacji, semantyczną strukturę kodu oraz wsparcie dla technologii asystujących.

6.4.1. Personalizacja i mechanizm trwałego zapisu ustawień

W górnym pasku nawigacyjnym zaimplementowano dedykowany panel dostępności, umożliwiający użytkownikom dostosowanie czytelności serwisu do własnych preferencji (Zdj. **16**). Panel oferuje trzy opcje dostosowania widoku:

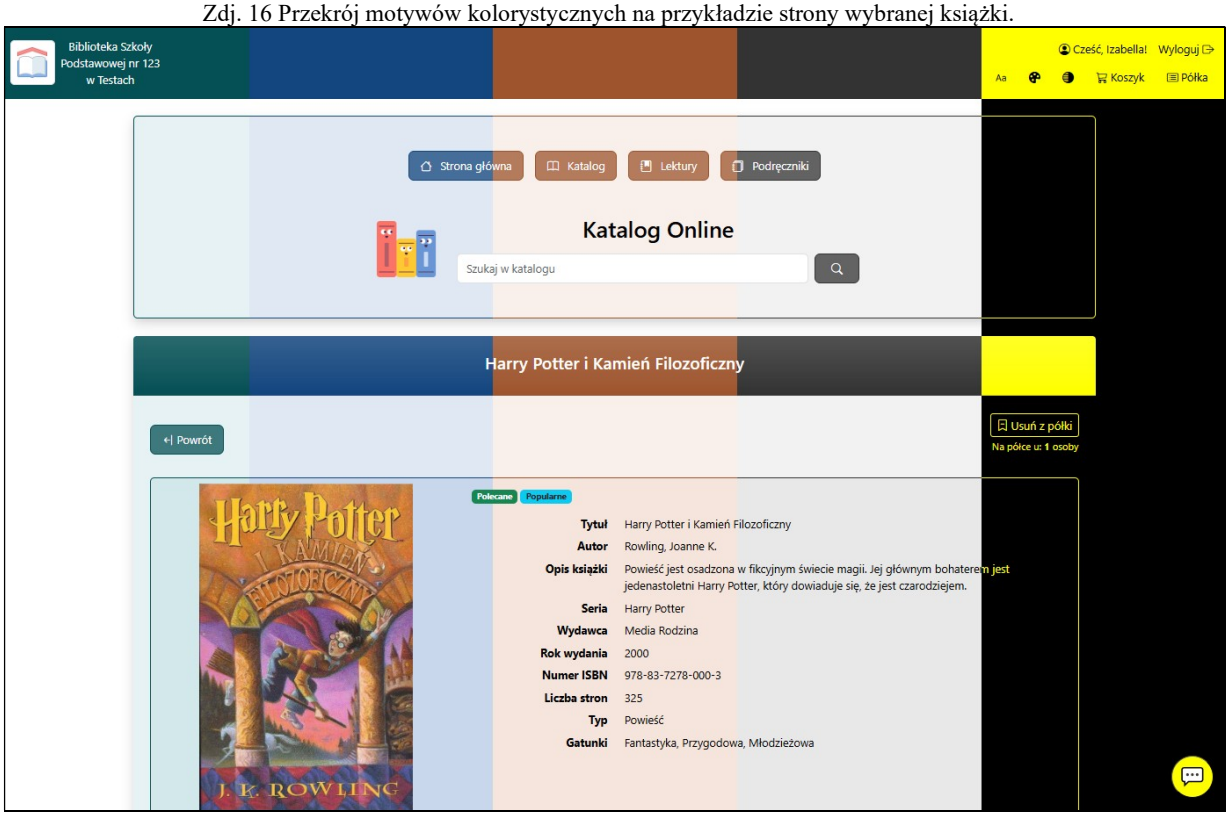
a) **Skalowanie typografii:** Trzystopniowa regulacja wielkości czcionki bazowej (mała 12px, normalna 16px, duża 20px),

b) **Wybór motywu:** Cztery motywy kolorystyczne (zielony, niebieski, brązowy, szary),

c) **Zarządzanie kontrastem:** Przełączanie na tryb wysokiego kontrastu.

Mechanizm ten wykorzystuje interfejs *IJSRuntime* do dynamicznego wstrzykiwania klas CSS oraz modyfikacji zmiennych CSS w czasie rzeczywistym, bez konieczności przeładowywania strony. Kluczowym aspektem jest trwałość tych ustawień, wybrane preferencje są serializowane i zapisywane w pamięci lokalnej (*LocalStorage*) przeglądarki, dzięki czemu są one automatycznie przywracane przy kolejnych wizytach użytkownika.

Zdj. 16 Przekrój motywów kolorystycznych na przykładzie strony wybranej książki.



Źródło 43 Opracowanie własne

6.4.2. Semantyka i technologie asystujące

Fundamentem dostępności jest poprawna hierarchia dokumentu HTML5. Wykorzystano semantyczne punkty orientacyjne (landmarks), takie jak *<header>, <main>, <search>* oraz *<footer>*, co umożliwia czytnikom ekranowym szybką nawigację między sekcjami. Elementy interaktywne pozbawione etykiet tekstowych (np. ikony) zostały opisane atrybutami *aria-label*, a elementy dekoracyjne ukryte za pomocą *aria-hidden="true"*.

Okno asystenta AI wykorzystuje atrybut *aria-live="polite"*, co pozwala na automatyczne odczytywanie nowych wiadomości przez lektora w sposób nieinwazyjny dla użytkownika. Z kolei komponent *<AuthorizeView>* dba o spójność nawigacji, usuwając z drzewa DOM elementy niedostępne dla niezalogowanych gości, co zapobiega dezorientacji podczas nawigacji klawiaturowej.

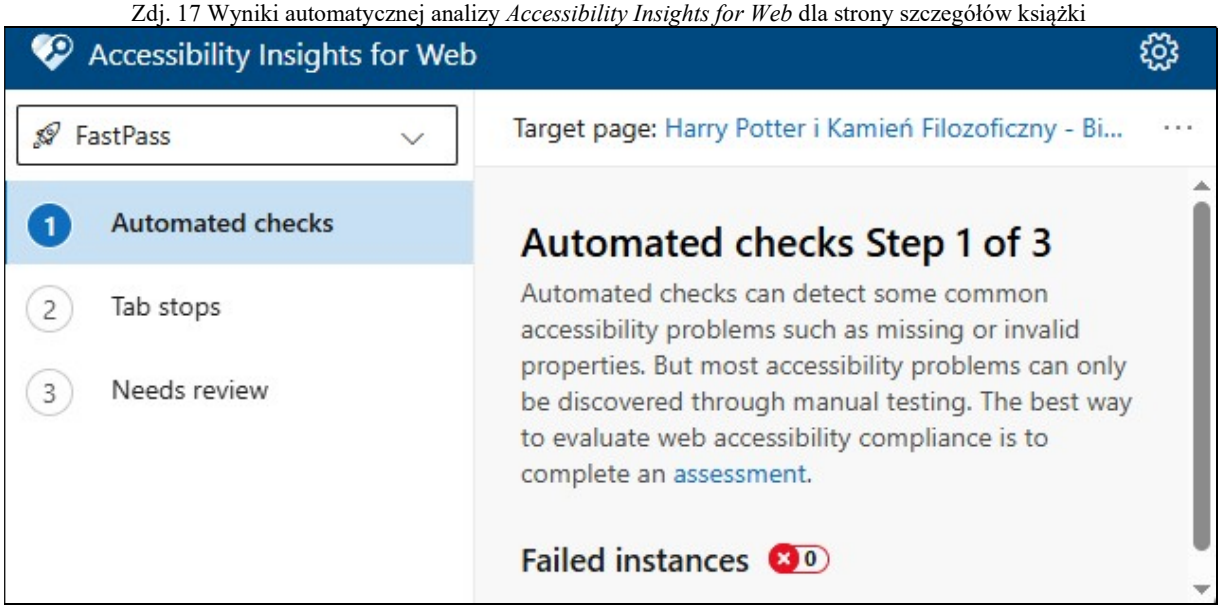
6.4.3. Weryfikacja i audyt dostępności z użyciem Accessibility Insights

W celu potwierdzenia jakości wdrożenia, przeprowadzono audyt dostępności przy użyciu narzędzia *Accessibility Insights for Web*[27] (**Zdj. 17**). Testy objęły kluczowe widoki systemu (Katalog, Strona Główna, Szczegóły Książki). Procedura weryfikacyjna składała się z dwóch etapów:

a) **Automatyczna analiza (FastPass):** Skaner kodu nie wykazał błędów krytycznych w zakresie kontrastu, etykiet ARIA ani struktury nagłówków na żadnej z testowanych podstron.

b) **Analiza nawigacji klawiaturowej (*Tab Stops*):** Wizualizacja ścieżki tabulacji potwierdziła logiczną kolejność przemieszczania się fokusu, umożliwiając pełną obsługę aplikacji wyłącznie za pomocą klawiatury.

Zdj. 17 Wyniki automatycznej analizy *Accessibility Insights for Web* dla strony szczegółów książki



Źródło 44 Wtyczka do przeglądarki Accessibility Insights

[27] www.accessibilityinsights.io